Salonia — Contexte complet du projet

Document destiné à un assistant IA (Claude) qui rejoint le projet. Mis à jour le 17/10/2026, après fusion des Modules 3 et 6 dans `main`. Dépôt GitHub : `Malekmouelh/Salonia`.

Instructions pour l'assistant

- Lis ce document, puis `README.md` et `AGENTS.md` à la racine du dépôt avant d'écrire du code.
- **Next.js 16.4** : les API ont changé par rapport à tes connaissances. En cas de doute, consulte `node_modules/next/dist/docs/`.
- Réutilise l'existant (modèles, DAL, i18n, composants UI, journal d'audit) au lieu de recréer. Respecte le style : TypeScript strict, commentaires courts en français.
- Ne modifie pas le code d'un autre module sans prévenir son responsable ; passe par une Pull Request.
- Ne lis, n'affiche et ne commite jamais le fichier `.env` ni aucun secret.

1. Le projet

Salonia est une application web progressive (PWA) de gestion de salons de beauté et de coiffure en Tunisie : multi-centres (un propriétaire, plusieurs salons dans plusieurs gouvernorats), bilingue français / arabe (RTL), avec de l'IA dans chaque module. Projet de groupe universitaire, découpé en modules selon un cahier des charges.

Module	Responsable	État
Socle : comptes, rôles, multi-centres, journal de sécurité, FR/AR, PWA	Groupe (réalisé par Malek)	Actif
Vitrine & personnalisation	—	À faire
Module 2 — Catalogue (services & packs)	Maya	À faire (schéma déjà posé)
Module 3 — Calendrier & réservations	Malek	Actif
Module 4 — Équipe	Amine	À faire
Matériel & stock	—	À faire
Module 6 — Paiements, factures & pilotage financier	Salsabil	Actif

2. Pile technique

Élément	Choix	Remarques importantes
Framework	Next.js 16.4 (App Router, Turbopack), React 19.3	`proxy.ts` remplace `middleware.ts`. `cacheComponents: true` et Partial Prefetching activés.
Langage	TypeScript strict	Types de routes générés : `PageProps<"/route">`, `RouteContext<…>`. Si `tsc` râle sur une nouvelle route : `npx next typegen`.
Base de données	MongoDB Atlas + Mongoose 9	Exigé par le professeur. **Pas de Prisma** (abandonné). Connexion unique : `connectDB()` dans `lib/db.ts`.
Validation	Zod 4	Toujours `safeParse` dans les Server Actions.

Auth	Session JWT (jose, HS256) dans un cookie httpOnly	Mots de passe bcryptjs. Pas de NextAuth.
Style	Tailwind CSS 4	Propriétés logiques (`ms-` , `pe-` , `start-` ...) pour l'arabe RTL. Thème sombre prune / or rosé, effet « glass ».
IA	SDK Anthropic, modèle `claude-opus-5-5`	Chaque fonction IA a un **mode hors ligne** (règles / mots-clés) si `ANTHROPIC_API_KEY` est vide.
PDF	pdf-lib	Factures, reçus, rapports (Module 6).
Tests	`node --test` via tsx	`npm test` exécute `lib/**/*.test.ts` .
Fuseau	Africa/Tunis (UTC+1, sans heure d'été)	Utiliser les helpers de `lib/booking/time.ts` (`toLocal` , `fromLocal` , `dayKey` ...), jamais l'heure locale du serveur.

3. Démarrage

```
npm install
cp .env.example .env        # puis remplir les valeurs (demander au groupe, ne jamais partager en clair)
npm run db:seed             # ATTENTION : efface et recrée toute la base de démo partagée
npm run db:seed:finance     # ajoute/régénère seulement les données financières
npm run dev                 # http://localhost:3000
npm test                    # tests automatisés
npx tsc --noEmit ; npx eslint . ; npx next build   # vérifications avant une PR
npm run db:local            # (option) MongoDB local sans installation, hors ligne
```

Variable `.env`	Rôle
`MONGODB_URI`	Chaîne Atlas `mongodb+srv://…` (ou `mongodb://127.0.0.1:27017` en local)
`MONGODB_DB`	Nom de la base (`salonia` par défaut ; possible : une base par membre, ex. `salonia_maya`)
`SESSION_SECRET`	Clé de signature des sessions
`DEMO_MODE`	`"true"` : boutons de comptes de démo sur /login
`ANTHROPIC_API_KEY`	Clé Claude ; vide = modes hors ligne
`CRON_SECRET`	Protège `/api/cron/reminders`

Comptes de démo (mot de passe : `Demo1234`)

Rôle	Identifiant
Propriétaire (3 centres : Tunis, Sousse, Djerba)	owner@salonia.tn
Gérant (Sousse)	manager@salonia.tn
Staff (Tunis)	staff@salonia.tn ou 22 345 678
Client	client@salonia.tn
Super admin	admin@salonia.tn

Salon de démo : « Salon Yasmine », page publique `/reserver/salon-yasmine` . Le seed crée 3 centres, 6 praticiennes, 11 services, 2 packs, ~320 clients, ~2900 rendez-vous et ~10 semaines de données financières.

4. Architecture du code

```
app/
  (auth)/login, register, forgot-password, reset-password
  dashboard/            page d'accueil + assistant, calendar, availability, clients, clients/[id],
                        finances, finances/caisse, finances/factures, finances/depenses
  reserver/[slug]       réservation publique (client, sans compte)
  reservation/[token]   gestion d'un RDV par le client (annuler / déplacer)
  paiement/[token]      paiement en ligne + /recu (PDF)
  api/                  auth/signout, cron/reminders, finance/export, finance/invoices/[id],
finance/receipts/[id]
  actions/              Server Actions : auth, centers, assistant, bookings, public-booking, chatbot,
finance
lib/
  db.ts  session.ts  dal.ts  audit.ts  enums.ts  definitions.ts  i18n-ar.ts
  models/   core.ts (Organization, Center, User, PasswordResetToken, AuthEvent)
            catalog.ts (Service, Pack)
            booking.ts (Customer, Booking, AvailabilityBlock, WaitlistEntry, Notification)
            finance.ts (Payment, Invoice, Expense, FinanceCounter)
  booking/  time, availability, create, noshow, reminders, messages, chatbot, format, types
  finance/  service, analytics, assistant, reports, pdf, documents, gateway, money, labels, *.test.ts
  data/     lectures pour les pages (bookings, customers, public-salon, organization, finance)
  ai/       assistant de configuration (offline-setup, setup-schema)
components/ ui, auth, dashboard, calendar, booking, clients, finance, assistant, i18n, pwa
scripts/    seed.ts, seed-finance.ts, local-db.ts
proxy.ts    redirection optimiste (cookie) vers /login pour /dashboard/*
```

5. Conventions à respecter

Sécurité et accès aux données

- `proxy.ts` ne fait qu'une vérification optimiste du cookie. **La vraie vérification** se fait côté serveur avec la DAL (`lib/dal.ts`) : `getCurrentUser()`, `requireRole(...roles)`, `accessibleCenterIds(user)`.
- Rôles : `SUPER_ADMIN`, `OWNER`, `MANAGER`, `STAFF`, `CLIENT`. Un utilisateur a `organizationId` et `centerIds[]` ; toute requête doit être filtrée par organisation **et** par centres accessibles.
- Chaque Server Action revérifie l'utilisateur et valide l'entrée avec Zod, même si l'interface masque déjà le bouton.
- Actions sensibles inscrites au journal : `logEvent(type, { userId, organizationId, detail })` (`lib/audit.ts`) ; ajouter le type dans `AUTH_EVENT_TYPES` (`lib/enums.ts`) et son libellé dans `components/dashboard/security-log.tsx`.
- Liens publics (réservation, paiement) : jeton aléatoire, seul son hash est stocké en base.

Next.js 16 (pièges déjà rencontrés)

- Une page qui lit des données à l'exécution (cookies, base, `new Date()`) doit être enveloppée dans `<Suspense>`. Une fonction de données qui utilise l'heure actuelle doit appeler `await connection()` (de `next/server`) au début, sinon erreur « unstable value new Date() while prerendering ».
- Données publiques mises en cache : `"use cache"` + `cacheLife(...)` + `cacheTag(...)`, invalidées avec `updateTag(...)` dans les Server Actions.
- Le linter interdit `setState` dans un `useEffect` : utiliser `useSyncExternalStore` ou dériver l'état.

Mongoose

- Les modèles utilisent le helper `model(name, schema)` qui réutilise `mongoose.models[name]` (rechargement à chaud). Ajouter `as const` pour les enums et typer les `populate<…>()`.
- Toujours `await connectDB()` avant une requête. Utiliser `.lean()` pour les lectures.

Bilingue français / arabe

- Les textes sont écrits en français dans le code. Côté client : `const { t } = useI18n()` puis `t("Texte")`, ou `<T>Texte</T>` (`components/i18n/i18n-provider.tsx`).
- Chaque nouveau texte doit avoir sa traduction dans le dictionnaire `lib/i18n-ar.ts` (clé = texte français exact).
- Mise en page : propriétés logiques Tailwind uniquement, pour que l'arabe s'affiche de droite à gauche.

IA

- Principe du cahier des charges : **l'IA propose, l'humain décide**. Aucune fonction IA ne modifie la base sans confirmation.
- Claude n'accède jamais directement à la base : il appelle des outils en lecture seule, déjà limités à l'organisation et aux centres de l'utilisateur.
- Toujours prévoir un repli hors ligne fonctionnel (démonstration sans clé API).

6. Module 3 — Calendrier et réservations (Malek)

Exigence	Où
F3.1 Calendrier par centre et praticien (jour / semaine / mois)	`/dashboard/calendar`
F3.2 Horaires, pauses, fermetures, blocage de créneaux	`/dashboard/availability`
F3.3 Réservation en ligne (services ou pack, praticien, créneau) sur 12 mois, max 6 services	`/reserver/[slug]`
F3.4 Rappels automatiques 24 h avant	`/api/cron/reminders` (Vercel Cron 07:00) + bouton manuel
F3.5 Annulations, reports, liste d'attente	calendrier + lien client `/reservation/[token]`
F3.6 Fiche client avec historique	`/dashboard/clients`

- **IA** : prédiction des absences (régression logistique par salon, AUC affichée, `lib/booking/noshow.ts`) ; créneaux à remplir (trous ≥ 45 min + liste d'attente) ; chatbot de réservation (Claude avec outils `find_slots` / `propose_booking` , repli mots-clés FR/AR).
- **Règles** : pas de double réservation (revérification à l'enregistrement), annulation / report en ligne jusqu'à 2 h avant, rappels seulement avec consentement.
- **Statuts** d'un RDV : `PENDING` , `CONFIRMED` , `COMPLETED` , `CANCELLED` , `NO_SHOW` . Sources : `ONLINE` , `STAFF` , `CHATBOT` , `WAITLIST` .
- Les clients ne créent **pas** de compte pour réserver (nom + téléphone suffisent). `/register` crée uniquement un compte propriétaire.
- WhatsApp : les boutons `wa.me` ouvrent le WhatsApp du salon ; les rappels automatiques sont mis en file dans la collection `notifications` (statut `QUEUED`) en attendant un fournisseur (Twilio / Meta / SMS).

7. Module 6 — Paiements, factures et pilotage financier (Salsabil)

Exigence	Où
F6.1 Caisse : espèces, carte, mobile, paiement fractionné, remise, acomptes, remboursements	`/dashboard/finances/caisse` (+ bouton « Encaisser » du calendrier)
F6.2 Paiement en ligne (passerelle de démonstration)	« Lien de paiement » → `/paiement/[token]`
F6.3 Factures et reçus PDF numérotés `FAC-2026-00001` / `REC-…`, TVA 19 % incluse	`/dashboard/finances/factures`, `/api/finance/*`
F6.4 Dépenses par centre et catégorie	`/dashboard/finances/depenses`
F6.5 Tableau de bord multi-centres	`/dashboard/finances`
F6.6 Export PDF et CSV	`/api/finance/export`

- **IA** : assistant d'analyse (6 outils en lecture seule), détection d'anomalies (médiane / écart absolu médian), prévision du CA du mois avec backtest.
- **Droits** : propriétaire = tout ; gérant = son centre, sans salaires ; staff = encaisser ses propres RDV, sans remise ni remboursement ; seul le propriétaire annule une facture.
- Montants TTC en dinars (3 décimales). Cartes de test : `4242 4242 4242 4242` acceptée, `4000 0000 0000 0002` refusée, expiration future au format MM/AA.

8. Contrats entre modules

Module	Contrat existant	Fichier
2 — Catalogue (Maya)	Collections `services` et `packs` (nom, catégorie, durée, prix, centres). Le Module 3 les lit pour réserver : ne pas renommer les champs existants, seulement en ajouter.	`lib/models/catalog.ts`
4 — Équipe (Amine)	Praticien = utilisateur `STAFF` rattaché au centre via `centerIds`. Les disponibilités en dépendent.	`lib/models/core.ts`, `lib/booking/availability.ts`
6 — Finances (Salsabil)	Lit `bookings` (`items`, `totalTnd`, statut) ; un RDV entièrement réglé passe à `COMPLETED`.	`lib/models/booking.ts`, `lib/finance/service.ts`
Notifications (socle)	Boîte d'envoi `notifications` (statut `QUEUED`) à brancher sur un fournisseur.	`lib/models/booking.ts`

Pour ajouter un module : nouveau fichier de modèles dans `lib/models/`, logique dans `lib/<module>/`, lectures dans `lib/data/`, actions dans `app/actions/`, pages sous `app/dashboard/<module>`, lien dans `components/dashboard/nav.tsx` (avec les rôles autorisés), passer le module à « Actif » dans `app/dashboard/page.tsx`, enrichir `scripts/seed.ts` et documenter dans `README.md`.

9. Méthode de travail Git du groupe

- Chaque membre travaille sur **sa propre branche** (ex. `Malek-Mouelhi`, `Salsabil-Hafsi`). Ne pas créer d'autres branches sans l'accord du membre.
- Intégration dans `main` uniquement par **Pull Request**, relue par un autre membre (revue croisée).
- Avant de commencer : `git checkout main`, `git pull`, `git checkout <ma-branche>`, `git merge main`.
- Avant une PR : `npx tsc --noEmit`, `npx eslint .`, `npm test`, `npx next build` doivent passer.

- Fins de ligne LF imposées (`.gitattributes`). `.env` est ignoré par Git ; seul `.env.example` (valeurs fictives) est versionné.
- La base Atlas est partagée : prévenir le groupe avant `npm run db:seed` (il efface tout), ou utiliser sa propre base via `MONGODB_DB` .

> **Sécurité** : ce document ne contient volontairement aucun mot de passe, clé ni chaîne de connexion. Les valeurs du `.env` se transmettent en privé entre membres du groupe, jamais dans un chat avec une IA, une capture d'écran ou un commit. Idéalement, un utilisateur Atlas par membre.